DATA MINING COURSE READINGS

Chapter 05: Regression Analysis Foundations

Curated Textbook Readings from Tan et al. · Matching Lecture: IE500_DM_05_Regression.pdf

REFERENCE TEXTBOOK

Introduction to Data Mining (2nd Edition, Global Edition)

Authors: Pang-Ning Tan, Michael Steinbach, Anuj Karpatne, Vipin Kumar | Pearson, 2019

Curated by: Ehsan Shahbazi | Shahid Beheshti University (SBU)

INCLUDED CHAPTER SECTIONS & TOPIC SCOPE

- **Section 6.6: Logistic Regression & GLM** — Book p. 446·452
  Linear formulation, odds ratio, logit link function, parameter learning, maximum likelihood
- **Section 6.10.3: Bias-Variance Decomposition** — Book p. 503·505
  Mathematical formulation of bias, variance, and irreducible error decomposition
- **Section 3.4: Overfitting & Complexity Penalties** — Book p. 168·176
  Model capacity, complexity penalties, training vs generalization error, regularization
- **Section 6.3: Instance-Based Learning Foundations** — Book p. 411·415
  Nearest neighbor principles applicable to distance-weighted regression estimation
- **Section 9.3.4: Regression Modeling for Outliers** — Book p. 736·739
  Statistical regression-based modeling and residual error evaluation

STUDENT STUDY GUIDE & SCOPE

While Tan et al. emphasizes discrete classification, the mathematical foundations of regression (Generalized Linear Models, Regularization, Overfitting, and Bias-Variance Tradeoff) are drawn from these core sections. For Python hands-on and advanced splines, refer to lecture slides, Géron (Ch. 4) and ISLR (Ch. 3, 7).

require more training instances for effectively learning the probability tables than the naïve Bayes classifier.

7. Although the sum-product algorithm provides computationally efficient techniques for performing inference over tree-like graphs, the complexity of the approach increase significantly when dealing with generic graphs of large sizes. In situations where exact inference is computationally infeasible, it is quite common to use approximate inference techniques.

6.6 Logistic Regression

The naïve Bayes and the Bayesian network classifiers described in the previous sections provide different ways of estimating the conditional probability of an instance $\mathbf{x}$ given class y, $P(\mathbf{x}|y)$. Such models are known as **probabilistic generative models**. Note that the conditional probability $P(\mathbf{x}|y)$ essentially describes the behavior of instances in the attribute space that are *generated* from class y. However, for the purpose of making predictions, we are finally interested in computing the posterior probability $P(y|\mathbf{x})$. For example, computing the following ratio of posterior probabilities is sufficient for inferring class labels in a binary classification problem:

$$\frac{P(y=1|\mathbf{x})}{P(y=0|\mathbf{x})}$$

This ratio is known as the **odds**. If this ratio is greater than 1, then $\mathbf{x}$ is classified as $y = 1$. Otherwise, it is assigned to class $y = 0$. Hence, one may simply learn a model of the odds based on the attribute values of training instances, without having to compute $P(\mathbf{x}|y)$ as an intermediate quantity in the Bayes theorem.

Classification models that directly assign class labels without computing class-conditional probabilities are called **discriminative models**. In this section, we present a probabilistic discriminative model known as **logistic regression**, which directly estimates the odds of a data instance $\mathbf{x}$ using its attribute values. The basic idea of logistic regression is to use a linear predictor, $z = \mathbf{w}^T\mathbf{x} + b$, for representing the odds of $\mathbf{x}$ as follows:

$$\frac{P(y=1|\mathbf{x})}{P(y=0|\mathbf{x})} = e^z = e^{\mathbf{w}^T\mathbf{x}+b}, \tag{6.37}$$

where $\mathbf{w}$ and b are the parameters of the model and $\mathbf{a}^T$ denotes the transpose of a vector $\mathbf{a}$. Note that if $\mathbf{w}^T\mathbf{x} + b > 0$, then $\mathbf{x}$ belongs to class 1 since its odds is greater than 1. Otherwise, $\mathbf{x}$ belongs to class 0.

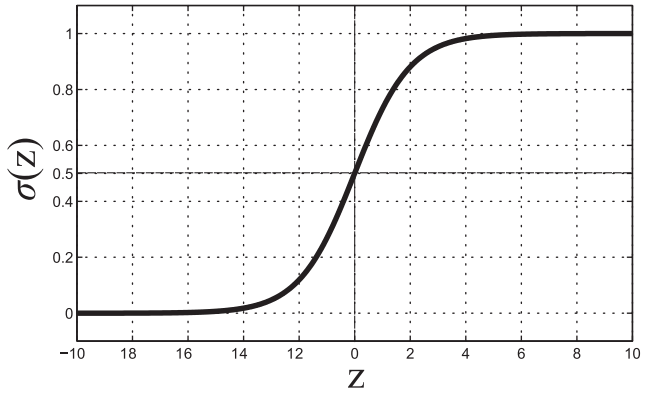


Figure 6.19. Plot of sigmoid (logistic) function, $\sigma(z)$.

Since $P(y = 0|\mathbf{x}) + P(y = 1|\mathbf{x}) = 1$, we can re-write Equation 6.37 as

$$\frac{P(y = 1|\mathbf{x})}{1 - P(y = 1|\mathbf{x})} = e^z.$$

This can be further simplified to express $P(y = 1|\mathbf{x})$ as a function of z.

$$P(y = 1|\mathbf{x}) = \frac{1}{1 + e^{-z}} = \sigma(z), \tag{6.38}$$

where the function $\sigma(.)$ is known as the logistic or sigmoid function. Figure 6.19 shows the behavior of the sigmoid function as we vary z. We can see that $\sigma(z) \geq 0.5$ only when $z \geq 0$. We can also derive $P(y = 0|\mathbf{x})$ using $\sigma(z)$ as follows:

$$P(y = 0|\mathbf{x}) = 1 - \sigma(z) = \frac{1}{1 + e^z} \tag{6.39}$$

Hence, if we have learned a suitable value of parameters $\mathbf{w}$ and b, we can use Equations 6.38 and 6.39 to estimate the posterior probabilities of any data instance $\mathbf{x}$ and determine its class label.

6.6.1 Logistic Regression as a Generalized Linear Model

Since the posterior probabilities are real-valued, their estimation using the previous equations can be viewed as solving a regression problem. In fact,

logistic regression belongs to a broader family of statistical regression models, known as **generalized linear models** (GLM). In these models, the target variable y is considered to be generated from a probability distribution $P(y|\mathbf{x})$, whose mean μ can be estimated using a link function $g(.)$ as follows:

$$g(\mu) = z = \mathbf{w}^T\mathbf{x} + b. \tag{6.40}$$

For binary classification using logistic regression, y follows a Bernoulli distribution (y can either be 0 or 1) and μ is equal to $P(y = 1|\mathbf{x})$. The link function $g(.)$ of logistic regression, called the logit function, can thus be represented as

$$g(\mu) = \log\Big(\frac{\mu}{1-\mu}\Big).$$

Depending on the choice of link function $g(.)$ and the form of probability distribution $P(y|\mathbf{x})$, GLMs are able to represent a broad family of regression models, such as linear regression and Poisson regression. They require different approaches for estimating their model parameters, $(\mathbf{w}, b)$. In this chapter, we will only discuss approaches for estimating the model parameters of logistic regression, although methods for estimating parameters of other types of GLMs are often similar (and sometimes even simpler). (See Bibliographic Notes for more details on GLMs.)

Note that even though logistic regression has relationships with regression models, it is a classification model since the computed posterior probabilities are eventually used to determine the class label of a data instance.

6.6.2 Learning Model Parameters

The parameters of logistic regression, $(\mathbf{w}, b)$, are estimated during training using a statistical approach known as the maximum likelihood estimation (MLE) method. This method involves computing the likelihood of observing the training data given $(\mathbf{w}, b)$, and then determining the model parameters $(\mathbf{w}^*, b^*)$ that yield maximum likelihood.

Let $D.train = \{(\mathbf{x_1}, y_1), (\mathbf{x_2}, y_2), \ldots, (\mathbf{x_n}, y_n)\}$ denote a set of n training instances, where y_i is a binary variable (0 or 1). For a given training instance $\mathbf{x_i}$, we can compute its posterior probabilities using Equations 6.38 and 6.39. We can then express the likelihood of observing y_i given $\mathbf{x_i}$, $\mathbf{w}$, and b as

$$\begin{aligned} P(y_i|\mathbf{x_i}, \mathbf{w}, b) &= P(y = 1|\mathbf{x_i})^{y_i} \times P(y = 0|\mathbf{x_i})^{1-y_i}, \\ &= (\sigma(z_i))^{y_i} \times (1 - \sigma(z_i))^{1-y_i}, \\ &= (\sigma(\mathbf{w}^T\mathbf{x_i} + b))^{y_i} \times (1 - \sigma(\mathbf{w}^T\mathbf{x_i} + b))^{1-y_i}, \end{aligned} \tag{6.41}$$

where $\sigma(.)$ is the sigmoid function as described above, Equation 6.41 basically means that the likelihood $P(y_i|\mathbf{x_i}, \mathbf{w}, b)$ is equal to $P(y = 1|\mathbf{x_i})$ when $y_i = 1$, and equal to $P(y = 0|\mathbf{x_i})$ when $y_i = 0$. The likelihood of all training instances, $\mathcal{L}(\mathbf{w}, b)$, can then be computed by taking the product of individual likelihoods (assuming independence among training instances) as follows:

$$\mathcal{L}(\mathbf{w}, b) = \prod_{i=1}^{n} P(y_i|\mathbf{x_i}, \mathbf{w}, b) = \prod_{i=1}^{n} P(y = 1|\mathbf{x_i})^{y_i} \times P(y = 0|\mathbf{x_i})^{1-y_i}. \quad (6.42)$$

The previous equation involves multiplying a large number of probability values, each of which are smaller than or equal to 1. Since this naïve computation can easily become numerically unstable when n is large, a more practical approach is to consider the negative logarithm (to base e) of the likelihood function, also known as the **cross entropy function**:

$$\begin{aligned} -\log \mathcal{L}(\mathbf{w}, b) &= -\sum_{i=1}^{n} y_i \log(P(y = 1|\mathbf{x_i})) + (1 - y_i) \log(P(y = 0|\mathbf{x_i})). \\ &= -\sum_{i=1}^{n} y_i \log(\sigma(\mathbf{w}^T\mathbf{x_i} + b)) + (1 - y_i) \log(1 - \sigma(\mathbf{w}^T\mathbf{x_i} + b)). \end{aligned}$$

The cross entropy is a loss function that measures how unlikely it is for the training data to be generated from the logistic regression model with parameters $(\mathbf{w}, b)$. Intuitively, we would like to find model parameters $(\mathbf{w}^*, b^*)$ that result in the lowest cross entropy, $-\log \mathcal{L}(\mathbf{w}^*, b^*)$.

$$\begin{aligned} (\mathbf{w}^*, b^*) &= \arg\min_{(\mathbf{w},b)} E(\mathbf{w}, b) \\ &= \arg\min_{(\mathbf{w},b)} -\log \mathcal{L}(\mathbf{w}, b), \end{aligned} \quad (6.43)$$

where $E(\mathbf{w}, b) = -\log \mathcal{L}(\mathbf{w}, b)$ is the loss function. It is worth emphasizing that $E(\mathbf{w}, b)$ is a convex function, i.e., any minima of $E(\mathbf{w}, b)$ will be a global minima. Hence, we can use any of the standard convex optimization techniques to solve Equation 6.43, which are mentioned in Appendix E. Here, we briefly describe the Newton-Raphson method that is commonly used for estimating the parameters of logistic regression. For ease of representation, we will use a single vector to describe $\tilde{\mathbf{w}} = (\mathbf{w}^T\ b)^T$, which is of size one greater than $\mathbf{w}$.

Similarly, we will consider the concatenated feature vector $\tilde{\mathbf{x}} = (\mathbf{x}^T\ 1)^T$, such that the linear predictor $z = \mathbf{w}^T\mathbf{x} + b$ can be succinctly written as $z = \tilde{\mathbf{w}}^T\tilde{\mathbf{x}}$. Also, the concatenation of all training labels, y_1 to y_n, will be represented as $\mathbf{y}$, the set consisting of $\sigma(z_1)$ to $\sigma(z_n)$ will be represented as $\boldsymbol{\sigma}$, and the concatenation of $\tilde{\mathbf{x}}_\mathbf{1}$ to $\tilde{\mathbf{x}}_\mathbf{n}$ will be represented as $\tilde{\mathbf{X}}$.

The Newton-Raphson is an iterative method for finding $\tilde{\mathbf{w}}^*$ that uses the following equation to update the model parameters at every iteration:

$$\tilde{\mathbf{w}}^{(new)} = \tilde{\mathbf{w}}^{(old)} - \mathbf{H}^{-1}\nabla E(\tilde{\mathbf{w}}), \tag{6.44}$$

where $\nabla E(\tilde{\mathbf{w}})$ and $\mathbf{H}$ are the first- and second-order derivatives of the loss function $E(\tilde{\mathbf{w}})$ with respect to $\tilde{\mathbf{w}}$, respectively. The key intuition behind Equation 6.44 is to move the model parameters in the direction of maximum gradient, such that $\tilde{\mathbf{w}}$ takes larger steps when $\nabla E(\tilde{\mathbf{w}})$ is large. When $\tilde{\mathbf{w}}$ arrives at a minima after some number of iterations, then $\nabla E(\tilde{\mathbf{w}})$ would become equal to 0 and thus result in convergence. Hence, we start with some initial values of $\tilde{\mathbf{w}}$ (either randomly assigned or set to 0) and use Equation 6.44 to iteratively update $\tilde{\mathbf{w}}$ till there are no significant changes in its value (beyond a certain threshold).

The first-order derivative of $E(\tilde{\mathbf{w}})$ is given by

$$\begin{aligned}
\nabla E(\tilde{\mathbf{w}}) &= -\sum_{i=1}^{n} y_i\tilde{\mathbf{x}}_\mathbf{i}(1-\sigma(\tilde{\mathbf{w}}^T\tilde{\mathbf{x}}_\mathbf{i})) - (1-y_i)\tilde{\mathbf{x}}_\mathbf{i}\sigma(\tilde{\mathbf{w}}^T\tilde{\mathbf{x}}_\mathbf{i}), \\
&= \sum_{i=1}^{n}(\sigma(\tilde{\mathbf{w}}^T\tilde{\mathbf{x}}_\mathbf{i}) - y_i)\tilde{\mathbf{x}}_\mathbf{i}, \\
&= \tilde{\mathbf{X}}(\boldsymbol{\sigma} - \mathbf{y}),
\end{aligned} \tag{6.45}$$

where we have used the fact that $d\sigma(z)/dz = \sigma(z)(1-\sigma(z))$. Using $\nabla E(\tilde{\mathbf{w}})$, we can compute the second-order derivative of $E(\tilde{\mathbf{w}})$ as

$$\begin{aligned}
\mathbf{H} = \nabla\nabla E(\tilde{\mathbf{w}}) &= \sum_{i=1}^{n}\sigma(\tilde{\mathbf{w}}^T\tilde{\mathbf{x}}_\mathbf{i})(1-\sigma(\tilde{\mathbf{w}}^T\tilde{\mathbf{x}}_\mathbf{i}))\tilde{\mathbf{x}}_\mathbf{i}\tilde{\mathbf{x}}_\mathbf{i}{}^T. \\
&= \tilde{\mathbf{X}}^T\mathbf{R}\tilde{\mathbf{X}},
\end{aligned} \tag{6.46}$$

where $\mathbf{R}$ is a diagonal matrix whose i^{th} diagonal element $\mathbf{R}_{ii} = \sigma_i(1-\sigma_i)$. We can now use the first- and second-order derivatives of $E(\tilde{\mathbf{w}})$ in Equation 6.44 to obtain the following update equation at the kth iteration:

$$\tilde{\mathbf{w}}^{(k+1)} = \tilde{\mathbf{w}}^{(k)} - (\tilde{\mathbf{X}}^T\mathbf{R}_k\tilde{\mathbf{X}})^{-1}\tilde{\mathbf{X}}^T(\boldsymbol{\sigma}_k - \mathbf{y}) \tag{6.47}$$

where the subscript k under $\mathbf{R}_k$ and $\boldsymbol{\sigma}_k$ refers to using $\tilde{\mathbf{w}}^{(k)}$ to compute both terms.

6.6.3 Characteristics of Logistic Regression

1. Logistic Regression is a discriminative model for classification that directly computes the poster probabilities without making any assumption about the class conditional probabilities. Hence, it is quite generic and can be applied in diverse applications. It can also be easily extended to multiclass classification, where it is known as **multinomial logistic regression**. However, its expressive power is limited to learning only linear decision boundaries.

2. Because there are different weights (parameters) for every attribute, the learned parameters of logistic regression can be analyzed to understand the relationships between attributes and class labels.

3. Because logistic regression does not involve computing densities and distances in the attribute space, it can work more robustly even in high-dimensional settings than distance-based methods such as nearest neighbor classifiers. However, the objective function of logistic regression does not involve any term relating to the complexity of the model. Hence, logistic regression does not provide a way to make a trade-off between model complexity and training performance, as compared to other classification models such as support vector machines. Nevertheless, variants of logistic regression can easily be developed to account for model complexity, by including appropriate terms in the objective function along with the cross entropy function.

4. Logistic regression can handle irrelevant attributes by learning weight parameters close to 0 for attributes that do not provide any gain in performance during training. It can also handle interacting attributes since the learning of model parameters is achieved in a joint fashion by considering the effects of all attributes together. Furthermore, if there are redundant attributes that are duplicates of each other, then logistic regression can learn equal weights for every redundant attribute, without degrading classification performance. However, the presence of a large number of irrelevant or redundant attributes in high-dimensional settings can make logistic regression susceptible to model overfitting.

5. Logistic regression cannot handle data instances with missing values, since the posterior probabilities are only computed by taking a weighted

sum of all the attributes. If there are missing values in a training instance, it can be discarded from the training set. However, if there are missing values in a test instance, then logistic regression would fail to predict its class label.

6.7 Artificial Neural Network (ANN)

Artificial neural networks (ANN) are powerful classification models that are able to learn highly complex and nonlinear decision boundaries purely from the data. They have gained widespread acceptance in several applications such as vision, speech, and language processing, where they have been repeatedly shown to outperform other classification models (and in some cases even human performance). Historically, the study of artificial neural networks was inspired by attempts to emulate biological neural systems. The human brain consists primarily of nerve cells called **neurons**, linked together with other neurons via strands of fiber called **axons**. Whenever a neuron is stimulated (e.g., in response to a stimuli), it transmits nerve activations via axons to other neurons. The receptor neurons collect these nerve activations using structures called **dendrites**, which are extensions from the cell body of the neuron. The strength of the contact point between a dendrite and an axon, known as a **synapse**, determines the connectivity between neurons. Neuroscientists have discovered that the human brain learns by changing the strength of the synaptic connection between neurons upon repeated stimulation by the same impulse.

The human brain consists of approximately 100 billion neurons that are inter-connected in complex ways, making it possible for us to learn new tasks and perform regular activities. Note that a single neuron only performs a simple modular function, which is to respond to the nerve activations coming from sender neurons connected at its dendrite, and transmit its activation to receptor neurons via axons. However, it is the composition of these simple functions that together is able to express complex functions. This idea is at the basis of constructing artificial neural networks.

Analogous to the structure of a human brain, an artificial neural network is composed of a number of processing units, called nodes, that are connected with each other via directed links. The nodes correspond to neurons that perform the basic units of computation, while the directed links correspond to connections between neurons, consisting of axons and dendrites. Further, the weight of a directed link between two neurons represents the strength of the synaptic connection between neurons. As in biological neural systems, the

made by the base classifiers $C_i(\mathbf{x})$:

$$C^*(\mathbf{x}) = f(C_1(\mathbf{x}), C_2(\mathbf{x}), \ldots, C_k(\mathbf{x})).$$

where f is the function that combines the ensemble responses. One simple approach for obtaining $C^*(\mathbf{x})$ is to take a majority vote of the individual predictions. An alternate approach is to take a *weighted* majority vote, where the weight of a base classifier denotes its accuracy or relevance.

Ensemble methods show the most improvement when used with **unstable classifiers**, i.e., base classifiers that are sensitive to minor perturbations in the training set, because of high model complexity. Although unstable classifiers may have a low *bias* in finding the optimal decision boundary, their predictions have a high *variance* for minor changes in the training set or model selection. This trade-off between bias and variance is discussed in detail in the next section. By aggregating the responses of multiple unstable classifiers, ensemble learning attempts to minimize their variance without worsening their bias.

6.10.3 Bias-Variance Decomposition

Bias-variance decomposition is a formal method for analyzing the generalization error of a predictive model. Although the analysis is slightly different for classification than regression, we first discuss the basic intuition of this decomposition by using an analogue of a regression problem.

Consider the illustrative task of reaching a target y by firing projectiles from a starting position $\mathbf{x}$, as shown in Figure 6.44. The target corresponds to the desired output at a test instance, while the starting position corresponds to its observed attributes. In this analogy, the projectile represents the model used for predicting the target using the observed attributes. Let $\hat{y}$ denote the

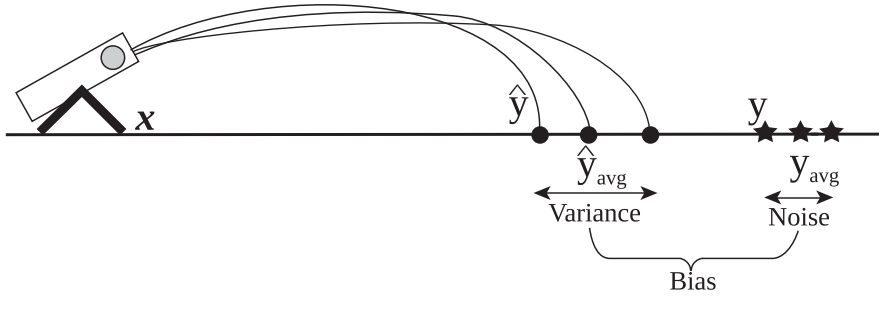


Figure 6.44. Bias-variance decomposition.

point where the projectile hits the ground, which is analogous of the prediction of the model.

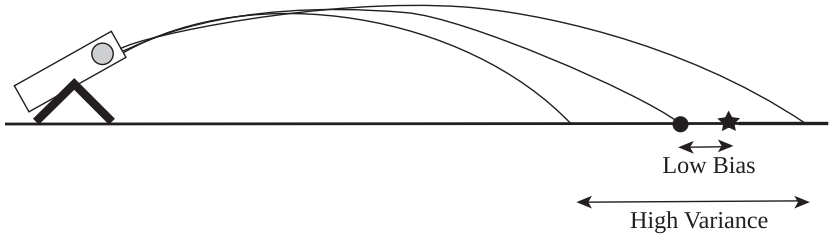
Ideally, we would like our predictions to be as close to the true target as possible. However, note that different trajectories of projectiles are possible based on differences in the training data or in the approach used for model selection. Hence, we can observe a **variance** in the predictions $\hat{y}$ over different runs of projectile. Further, the target in our example is not fixed but has some freedom to move around, resulting in a **noise** component in the true target. This can be understood as the non-deterministic nature of the output variable, where the same set of attributes can have different output values. Let $\hat{y}_{avg}$ represent the *average* prediction of the projectile over multiple runs, and y_{avg} denote the average target value. The difference between $\hat{y}_{avg}$ and y_{avg} is known as the **bias** of the model.

In the context of classification, it can be shown that the generalization error of a classification model m can be decomposed into terms involving the bias, variance, and noise components of the model in the following way:

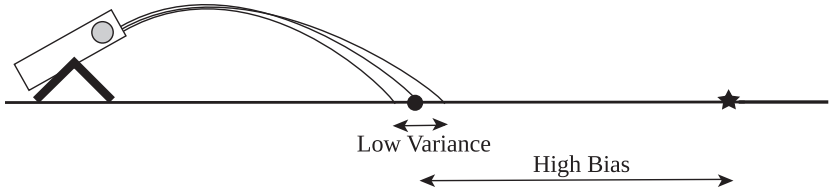
$$gen.error(m) = c_1 \times noise + bias(m) + c_2 \times variance(m)$$

where c_1 and c_2 are constants that depend on the characteristics of training and test sets. Note that while the noise term is intrinsic to the target class, the bias and variance terms depend on the choice of the classification model. The bias of a model represents how close the average prediction of the model is to the average target. Models that are able to learn complex decision boundaries, e.g., models produced by k-nearest neighbor and multi-layer ANN, generally show low bias. The variance of a model captures the *stability* of its predictions in response to minor perturbations in the training set or the model selection approach.

We can say that a model shows better generalization performance if it has a *lower bias* and *lower variance*. However, if the complexity of a model is high but the training size is small, we generally expect to see a lower bias but *higher variance*, resulting in the phenomena of overfitting. This phenomena is pictorially represented in Figure 6.45(a). On the other hand, an overly simplistic model that suffers from underfitting may show a lower variance but would suffer from a *high bias*, as shown in Figure 6.45(b). Hence, the trade-off between bias and variance provides a useful way for interpreting the effects of underfitting and overfitting on the generalization performance of a model.



(a) Phenomena of Overfitting.



(b) Phenomena of Underfitting.

Figure 6.45. Plots showing the behavior of two-dimensional solutions with constant L_2 and L_1 norms.

The bias-variance trade-off can be used to explain why ensemble learning improves the generalization performance of unstable classifiers. If a base classifier show low bias but high variance, it can become susceptible to overfitting, as even a small change in the training set will result in different predictions. However, by combining the responses of multiple base classifiers, we can expect to reduce the overall variance. Hence, ensemble learning methods show better performance primarily by lowering the variance in the predictions, although they can even help in reducing the bias. One of the simplest approaches for combining predictions and reducing their variance is to compute their average. This forms the basis of the bagging method, described in the following subsection.

6.10.4 Bagging

Bagging, which is also known as bootstrap aggregating, is a technique that repeatedly samples (with replacement) from a data set according to a uniform probability distribution. Each bootstrap sample has the same size as the original data. Because the sampling is done with replacement, some instances may appear several times in the same training set, while others may be

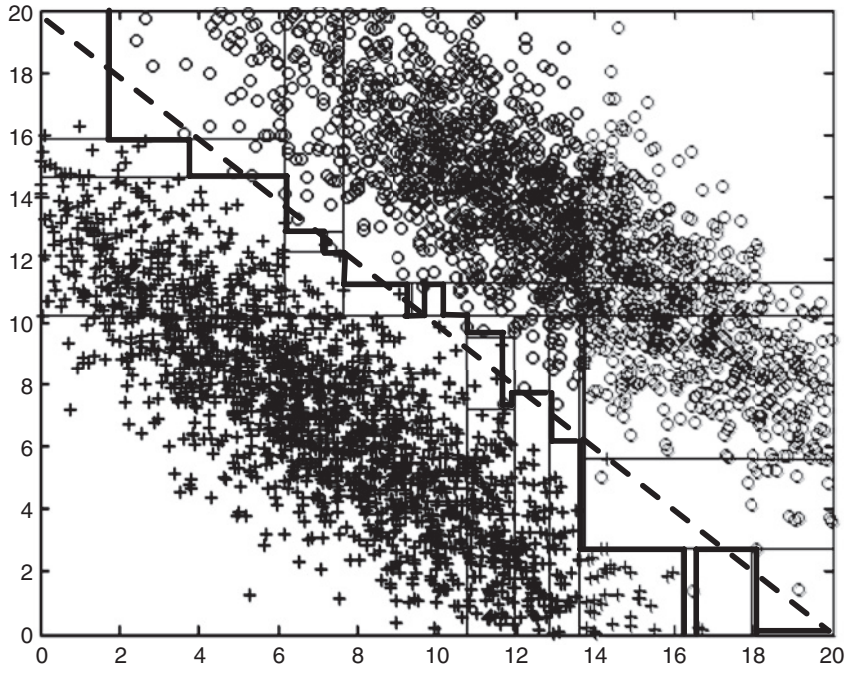


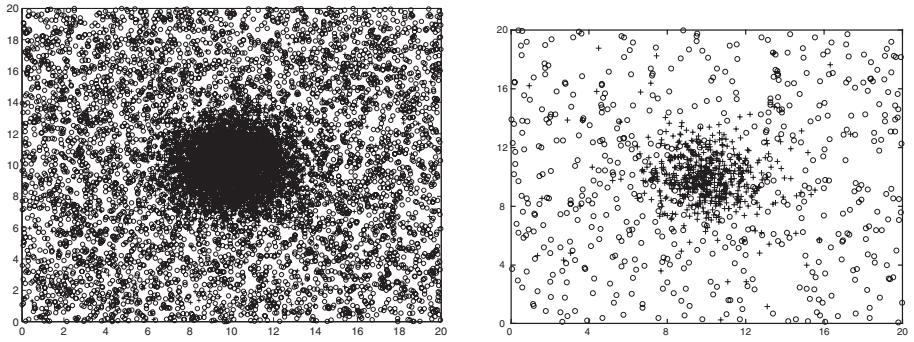
Figure 3.21. Example of data set that cannot be partitioned optimally using a decision tree with single attribute test conditions. The true decision boundary is shown by the dashed line.

Although an oblique decision tree is more expressive and can produce more compact trees, finding the optimal test condition is computationally more expensive.

9. **Choice of Impurity Measure:** It should be noted that the choice of impurity measure often has little effect on the performance of decision tree classifiers since many of the impurity measures are quite consistent with each other, as shown in Figure 3.11 on page 149. Instead, the strategy used to prune the tree has a greater impact on the final tree than the choice of impurity measure.

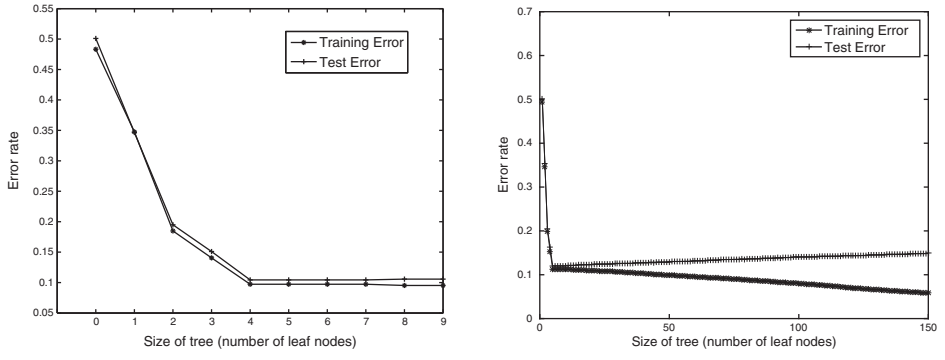
3.4 Model Overfitting

Methods presented so far try to learn classification models that show the lowest error on the training set. However, as we will show in the following example, even if a model fits well over the training data, it can still show poor generalization performance, a phenomenon known as model overfitting.



(a) Example of a 2-D data. (b) Training set using 10% data.

Figure 3.22. Examples of training and test sets of a two-dimensional classification problem.



(a) Varying tree size from 1 to 8. (b) Varying tree size from 1 to 150.

Figure 3.23. Effect of varying tree size (number of leaf nodes) on training and test errors.

Example 3.5. [Overfitting and Underfitting of Decision Trees] Consider the two-dimensional data set shown in Figure 3.22(a). The data set contains instances that belong to two separate classes, represented as + and o, respectively, where each class has 5400 instances. All instances belonging to the o class were generated from a uniform distribution. For the + class, 5000 instances were generated from a Gaussian distribution centered at (10,10) with unit variance, while the remaining 400 instances were sampled from the same uniform distribution as the o class. We can see from Figure 3.22(a) that

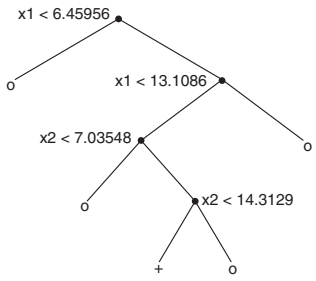
the + class can be largely distinguished from the o class by drawing a circle of appropriate size centered at (10,10). To learn a classifier using this two-dimensional data set, we randomly sampled 10% of the data for training and used the remaining 90% for testing. The training set, shown in Figure 3.22(b), looks quite representative of the overall data. We used Gini index as the impurity measure to construct decision trees of increasing sizes (number of leaf nodes), by recursively expanding a node into child nodes till every leaf node was pure, as described in Section 3.3.4.

Figure 3.23(a) shows changes in the training and test error rates as the size of the tree varies from 1 to 8. Both error rates are initially large when the tree has only one or two leaf nodes. This situation is known as **model underfitting**. Underfitting occurs when the learned decision tree is too simplistic, and thus, incapable of fully representing the true relationship between the attributes and the class labels. As we increase the tree size from 1 to 8, we can observe two effects. First, both the error rates decrease since larger trees are able to represent more complex decision boundaries. Second, the training and test error rates are quite close to each other, which indicates that the performance on the training set is fairly representative of the generalization performance. As we further increase the size of the tree from 8 to 150, the training error continues to steadily decrease till it eventually reaches zero, as shown in Figure 3.23(b). However, in a striking contrast, the test error rate ceases to decrease any further beyond a certain tree size, and then it begins to increase. The training error rate thus grossly under-estimates the test error rate once the tree becomes too large. Further, the gap between the training and test error rates keeps on widening as we increase the tree size. This behavior, which may seem counter-intuitive at first, can be attributed to the phenomena of **model overfitting**.

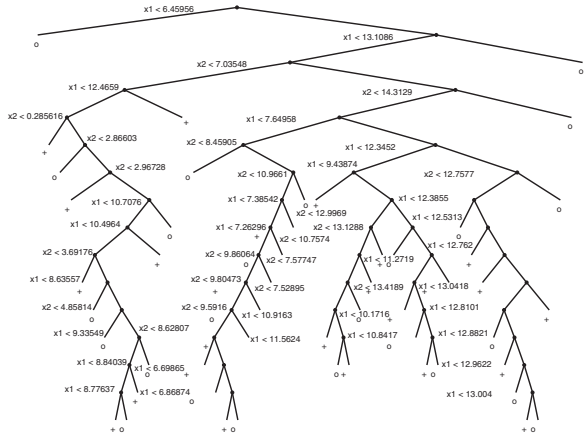
■

3.4.1 Reasons for Model Overfitting

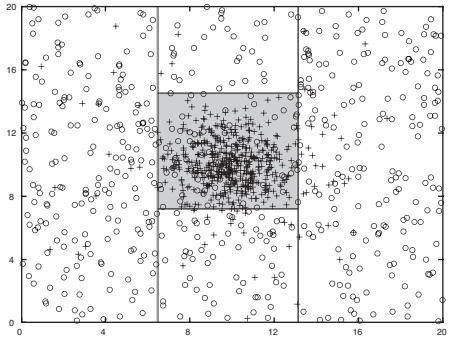
Model overfitting is the phenomena where, in the pursuit of minimizing the training error rate, an overly complex model is selected that captures specific patterns in the training data but fails to learn the *true* nature of relationships between attributes and class labels in the overall data. To illustrate this, Figure 3.24 shows decision trees and their corresponding decision boundaries (shaded rectangles represent regions assigned to the + class) for two trees of sizes 5 and 50. We can see that the decision tree of size 5 appears quite simple and its decision boundaries provide a reasonable approximation to the ideal decision boundary, which in this case corresponds to a circle centered around



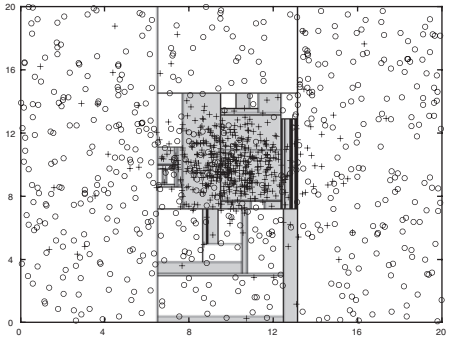
(a) Decision tree with 5 leaf nodes.



(b) Decision tree with 50 leaf nodes.



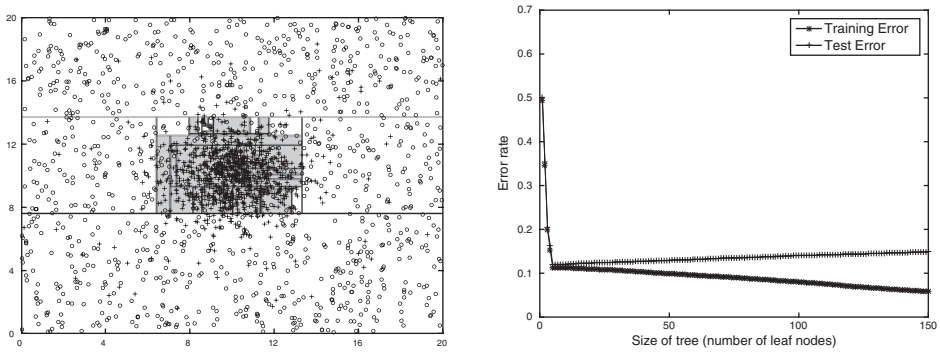
(c) Decision boundary for tree with 5 leaf nodes.



(d) Decision boundary for tree with 50 leaf nodes.

Figure 3.24. Decision trees with different model complexities.

the Gaussian distribution at (10, 10). Although its training and test error rates are non-zero, they are very close to each other, which indicates that the patterns learned in the training set should generalize well over the test set. On the other hand, the decision tree of size 50 appears much more complex than the tree of size 5, with complicated decision boundaries. For example, some of its shaded rectangles (assigned the + class) attempt to cover narrow regions in the input space that contain only one or two + training instances. Note that



(a) Decision boundary for tree with 50 leaf nodes using 20% data for training.

(b) Training and test error rates using 20% data for training.

Figure 3.25. Performance of decision trees using 20% data for training (twice the original training size).

the prevalence of + instances in such regions is highly specific to the training set, as these regions are mostly dominated by − instances in the overall data. Hence, in an attempt to perfectly fit the training data, the decision tree of size 50 starts fine tuning itself to specific patterns in the training data, leading to poor performance on an independently chosen test set.

There are a number of factors that influence model overfitting. In the following, we provide brief descriptions of two of the major factors: limited training size and high model complexity. Though they are not exhaustive, the interplay between them can help explain most of the common model overfitting phenomena in real-world applications.

Limited Training Size

Note that a training set consisting of a finite number of instances can only provide a limited representation of the overall data. Hence, it is possible that the patterns learned from a training set do not fully represent the true patterns in the overall data, leading to model overfitting. In general, as we increase the size of a training set (number of training instances), the patterns learned from the training set start resembling the true patterns in the overall data. Hence, the effect of overfitting can be reduced by increasing the training size, as illustrated in the following example.

Example 3.6. [Effect of Training Size] Suppose that we use twice the number of training instances than what we had used in the experiments conducted in Example 3.5. Specifically, we use 20% data for training and use the remainder for testing. Figure 3.25(b) shows the training and test error rates as the size of the tree is varied from 1 to 150. There are two major differences in the trends shown in this figure and those shown in Figure 3.23(b) (using only 10% of the data for training). First, even though the training error rate decreases with increasing tree size in both figures, its rate of decrease is much smaller when we use twice the training size. Second, for a given tree size, the gap between the training and test error rates is much smaller when we use twice the training size. These differences suggest that the patterns learned using 20% of data for training are more generalizable than those learned using 10% of data for training.

Figure 3.25(a) shows the decision boundaries for the tree of size 50, learned using 20% of data for training. In contrast to the tree of the same size learned using 10% data for training (see Figure 3.24(d)), we can see that the decision tree is not capturing specific patterns of noisy + instances in the training set. Instead, the high model complexity of 50 leaf nodes is being effectively used to learn the boundaries of the + instances centered at (10, 10).

■

High Model Complexity

Generally, a more complex model has a better ability to represent complex patterns in the data. For example, decision trees with larger number of leaf nodes can represent more complex decision boundaries than decision trees with fewer leaf nodes. However, an overly complex model also has a tendency to learn specific patterns in the training set that do not generalize well over unseen instances. Models with high complexity should thus be judiciously used to avoid overfitting.

One measure of model complexity is the number of "parameters" that need to be inferred from the training set. For example, in the case of decision tree induction, the attribute test conditions at internal nodes correspond to the parameters of the model that need to be inferred from the training set. A decision tree with larger number of attribute test conditions (and consequently more leaf nodes) thus involves more "parameters" and hence is more complex.

Given a class of models with a certain number of parameters, a learning algorithm attempts to select the best combination of parameter values that maximizes an evaluation metric (e.g., accuracy) over the training set. If the number of parameter value combinations (and hence the complexity) is large,

the learning algorithm has to select the best combination from a large number of possibilities, using a limited training set. In such cases, there is a high chance for the learning algorithm to pick a *spurious* combination of parameters that maximizes the evaluation metric just by random chance. This is similar to the **multiple comparisons problem** (also referred as multiple testing problem) in statistics.

As an illustration of the multiple comparisons problem, consider the task of predicting whether the stock market will rise or fall in the next ten trading days. If a stock analyst simply makes random guesses, the probability that her prediction is correct on any trading day is 0.5. However, the probability that she will predict correctly at least nine out of ten times is

$$\frac{\binom{10}{9} + \binom{10}{10}}{2^{10}} = 0.0107,$$

which is extremely low.

Suppose we are interested in choosing an investment advisor from a pool of 200 stock analysts. Our strategy is to select the analyst who makes the most number of correct predictions in the next ten trading days. The flaw in this strategy is that even if all the analysts make their predictions in a random fashion, the probability that at least one of them makes at least nine correct predictions is

$$1 - (1 - 0.0107)^{200} = 0.8847,$$

which is very high. Although each analyst has a low probability of predicting at least nine times correctly, considered together, we have a high probability of finding at least one analyst who can do so. However, there is no guarantee in the future that such an analyst will continue to make accurate predictions by random guessing.

How does the multiple comparisons problem relate to model overfitting? In the context of learning a classification model, each combination of parameter values corresponds to an analyst, while the number of training instances corresponds to the number of days. Analogous to the task of selecting the best analyst who makes the most accurate predictions on consecutive days, the task of a learning algorithm is to select the best combination of parameters that results in the highest accuracy on the training set. If the number of parameter combinations is large but the training size is small, it is highly likely for the learning algorithm to choose a spurious parameter combination that provides high training accuracy just by random chance. In the following example, we illustrate the phenomena of overfitting due to multiple comparisons in the context of decision tree induction.

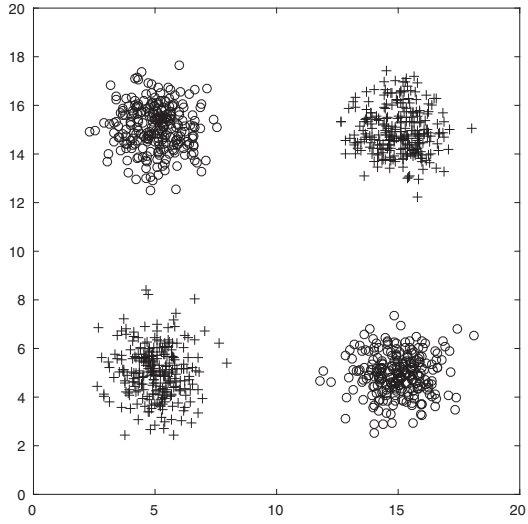
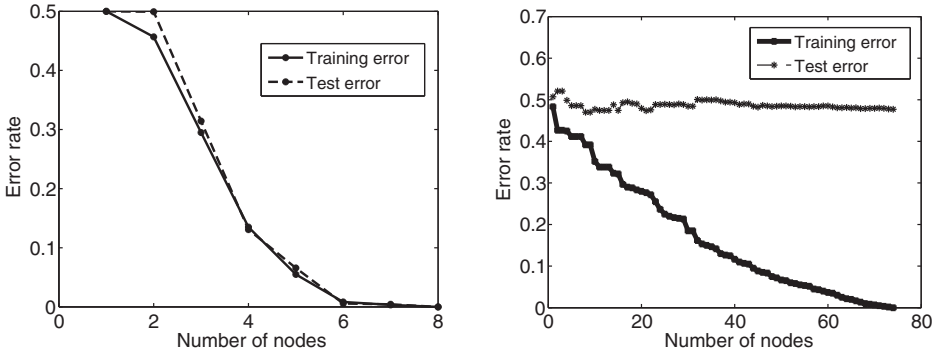


Figure 3.26. Example of a two-dimensional (X-Y) data set.

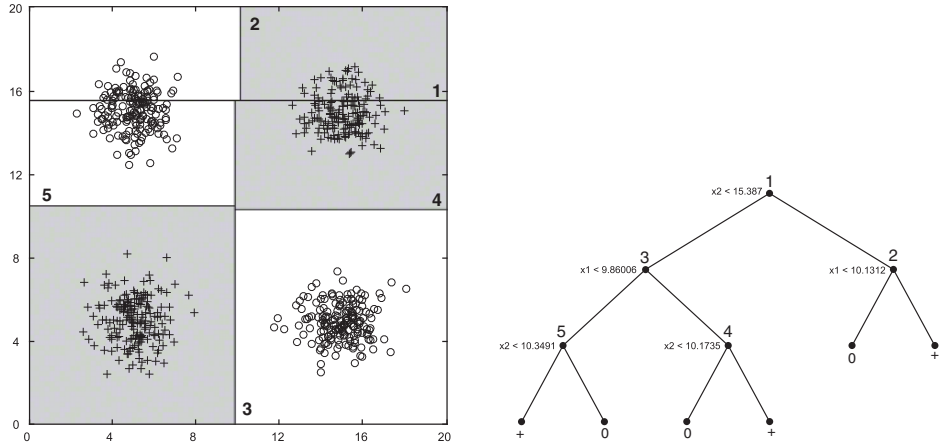


(a) Using X and Y attributes only.

(b) After adding 100 irrelevant attributes.

Figure 3.27. Training and test error rates illustrating the effect of multiple comparisons problem on model overfitting.

Example 3.7. [Multiple Comparisons and Overfitting] Consider the two-dimensional data set shown in Figure 3.26 containing 500 + and 500 o instances, which is similar to the data shown in Figure 3.19. In this data set, the distributions of both classes are well-separated in the two-dimensional (X-Y) attribute space, but none of the two attributes (X or Y) are sufficiently informative to be used alone for separating the two classes. Hence, splitting



(a) Decision boundary for tree with 6 leaf nodes.

(b) Decision tree with 6 leaf nodes.

Figure 3.28. Decision tree with 6 leaf nodes using X and Y as attributes. Splits have been numbered from 1 to 5 in order of other occurrence in the tree.

the data set based on any value of an X or Y attribute will provide close to zero reduction in an impurity measure. However, if X and Y attributes are used together in the splitting criterion (e.g., splitting X at 10 *and* Y at 10), the two classes can be effectively separated.

Figure 3.27(a) shows the training and test error rates for learning decision trees of varying sizes, when 30% of the data is used for training and the remainder of the data for testing. We can see that the two classes can be separated using a small number of leaf nodes. Figure 3.28 shows the decision boundaries for the tree with six leaf nodes, where the splits have been numbered according to their order of appearance in the tree. Note that the even though splits 1 and 3 provide trivial gains, their consequent splits (2, 4, and 5) provide large gains, resulting in effective discrimination of the two classes.

Assume we add 100 irrelevant attributes to the two-dimensional X-Y data. Learning a decision tree from this resultant data will be challenging because the number of candidate attributes to choose for splitting at every internal node will increase from two to 102. With such a large number of candidate attribute test conditions to choose from, it is quite likely that spurious attribute test conditions will be selected at internal nodes because of the multiple comparisons problem. Figure 3.27(b) shows the training and test error rates after adding 100 irrelevant attributes to the training set. We can see that the

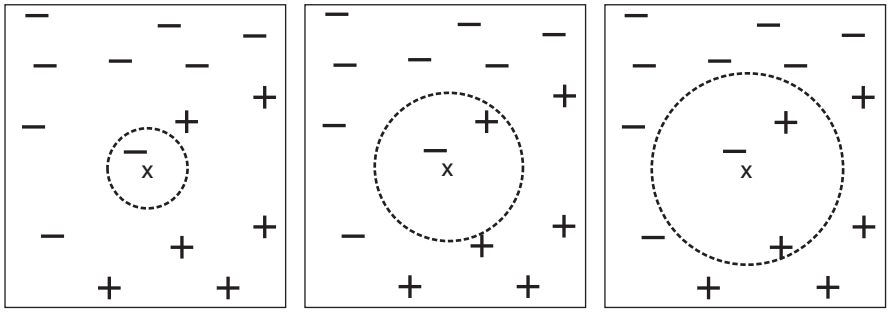
6.3 Nearest Neighbor Classifiers

The classification framework shown in Figure 3.3 involves a two-step process: (1) an inductive step for constructing a classification model from data, and (2) a deductive step for applying the model to test examples. Decision tree and rule-based classifiers are examples of **eager learners** because they are designed to learn a model that maps the input attributes to the class label as soon as the training data becomes available. An opposite strategy would be to delay the process of modeling the training data until it is needed to classify the test instances. Techniques that employ this strategy are known as **lazy learners**. An example of a lazy learner is the **Rote classifier**, which memorizes the entire training data and performs classification only if the attributes of a test instance match one of the training examples exactly. An obvious drawback of this approach is that some test instances may not be classified because they do not match any training example.

One way to make this approach more flexible is to find all the training examples that are relatively similar to the attributes of the test instances. These examples, which are known as **nearest neighbors**, can be used to determine the class label of the test instance. The justification for using nearest neighbors is best exemplified by the following saying: *"If it walks like a duck, quacks like a duck, and looks like a duck, then it's probably a duck."* A nearest neighbor classifier represents each example as a data point in a d-dimensional space, where d is the number of attributes. Given a test instance, we compute its proximity to the training instances according to one of the proximity measures described in Section 2.4 on page 91. The k-nearest neighbors of a given test instance z refer to the k training examples that are closest to z.

Figure 6.6 illustrates the 1-, 2-, and 3-nearest neighbors of a test instance located at the center of each circle. The instance is classified based on the class labels of its neighbors. In the case where the neighbors have more than one label, the test instance is assigned to the majority class of its nearest neighbors. In Figure 6.6(a), the 1-nearest neighbor of the instance is a negative example. Therefore the instance is assigned to the negative class. If the number of nearest neighbors is three, as shown in Figure 6.6(c), then the neighborhood contains two positive examples and one negative example. Using the majority voting scheme, the instance is assigned to the positive class. In the case where there is a tie between the classes (see Figure 6.6(b)), we may randomly choose one of them to classify the data point.

The preceding discussion underscores the importance of choosing the right value for k. If k is too small, then the nearest neighbor classifier may be



(a) 1-nearest neighbor (b) 2-nearest neighbor (c) 3-nearest neighbor

Figure 6.6. The 1-, 2-, and 3-nearest neighbors of an instance.

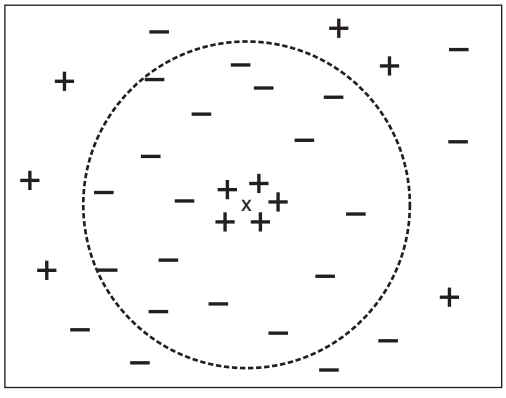


Figure 6.7. k-nearest neighbor classification with large k.

susceptible to overfitting due to noise, i.e., mislabeled examples in the training data. On the other hand, if k is too large, the nearest neighbor classifier may misclassify the test instance because its list of nearest neighbors includes training examples that are located far away from its neighborhood (see Figure 6.7).

6.3.1 Algorithm

A high-level summary of the nearest neighbor classification method is given in Algorithm 6.2. The algorithm computes the distance (or similarity) between each test instance $z = (\mathbf{x}', y')$ and all the training examples $(\mathbf{x}, y) \in D$ to determine its nearest neighbor list, D_z. Such computation can be costly if the number of training examples is large. However, efficient indexing techniques

Algorithm 6.2 The k-nearest neighbor classifier.

1: Let k be the number of nearest neighbors and D be the set of training examples.
2: **for** each test instance $z = (\mathbf{x}', y')$ **do**
3: Compute $d(\mathbf{x}', \mathbf{x})$, the distance between z and every example, $(\mathbf{x}, y) \in D$.
4: Select $D_z \subseteq D$, the set of k closest training examples to z.
5: $y' = \underset{v}{\operatorname{argmax}} \sum_{(\mathbf{x}_i, y_i) \in D_z} I(v = y_i)$
6: **end for**

are available to reduce the computation needed to find the nearest neighbors of a test instance.

Once the nearest neighbor list is obtained, the test instance is classified based on the majority class of its nearest neighbors:

$$\text{Majority Voting: } y' = \underset{v}{\operatorname{argmax}} \sum_{(\mathbf{x}_i, y_i) \in D_z} I(v = y_i), \tag{6.5}$$

where v is a class label, y_i is the class label for one of the nearest neighbors, and $I(\cdot)$ is an indicator function that returns the value 1 if its argument is true and 0 otherwise.

In the majority voting approach, every neighbor has the same impact on the classification. This makes the algorithm sensitive to the choice of k, as shown in Figure 6.6. One way to reduce the impact of k is to weight the influence of each nearest neighbor $\mathbf{x}_i$ according to its distance: $w_i = 1/d(\mathbf{x}', \mathbf{x}_i)^2$. As a result, training examples that are located far away from z have a weaker impact on the classification compared to those that are located close to z. Using the distance-weighted voting scheme, the class label can be determined as follows:

$$\text{Distance-Weighted Voting: } y' = \underset{v}{\operatorname{argmax}} \sum_{(\mathbf{x}_i, y_i) \in D_z} w_i \times I(v = y_i). \tag{6.6}$$

6.3.2 Characteristics of Nearest Neighbor Classifiers

1. Nearest neighbor classification is part of a more general technique known as instance-based learning, which does not build a global model, but rather uses the training examples to make predictions for a test instance. (Thus, such classifiers are often said to be "model free.") Such algorithms require a proximity measure to determine the similarity or distance between instances and a classification function that returns the predicted class of a test instance based on its proximity to other instances.

2. Although lazy learners, such as nearest neighbor classifiers, do not require model building, classifying a test instance can be quite expensive because we need to compute the proximity values individually between the test and training examples. In contrast, eager learners often spend the bulk of their computing resources for model building. Once a model has been built, classifying a test instance is extremely fast.

3. Nearest neighbor classifiers make their predictions based on local information. (This is equivalent to building a local model for each test instance.) By contrast, decision tree and rule-based classifiers attempt to find a global model that fits the entire input space. Because the classification decisions are made locally, nearest neighbor classifiers (with small values of k) are quite susceptible to noise.

4. Nearest neighbor classifiers can produce decision boundaries of arbitrary shape. Such boundaries provide a more flexible model representation compared to decision tree and rule-based classifiers that are often constrained to rectilinear decision boundaries. The decision boundaries of nearest neighbor classifiers also have high variability because they depend on the composition of training examples in the local neighborhood. Increasing the number of nearest neighbors may reduce such variability.

5. Nearest neighbor classifiers have difficulty handling missing values in both the training and test sets since proximity computations normally require the presence of all attributes. Although, the subset of attributes present in two instances can be used to compute a proximity, such an approach may not produce good results since the proximity measures may be different for each pair of instances and thus hard to compare.

6. Nearest neighbor classifiers can handle the presence of interacting attributes, i.e., attributes that have more predictive power taken in combination then by themselves, by using appropriate proximity measures that can incorporate the effects of multiple attributes together.

7. The presence of irrelevant attributes can distort commonly used proximity measures, especially when the number of irrelevant attributes is large. Furthermore, if there are a large number of redundant attributes that are highly correlated with each other, then the proximity measure can be overly biased toward such attributes, resulting in improper estimates of distance. Hence, the presence of irrelevant and redundant attributes can adversely affect the performance of nearest neighbor classifiers.

8. Nearest neighbor classifiers can produce wrong predictions unless the appropriate proximity measure and data preprocessing steps are taken. For example, suppose we want to classify a group of people based on attributes such as height (measured in meters) and weight (measured in pounds). The height attribute has a low variability, ranging from 1.5 m to 1.85 m, whereas the weight attribute may vary from 90 lb. to 250 lb. If the scale of the attributes are not taken into consideration, the proximity measure may be dominated by differences in the weights of a person.

6.4 Naïve Bayes Classifier

Many classification problems involve uncertainty. First, the observed attributes and class labels may be unreliable due to imperfections in the measurement process, e.g., due to the limited preciseness of sensor devices. Second, the set of attributes chosen for classification may not be fully representative of the target class, resulting in uncertain predictions. To illustrate this, consider the problem of predicting a person's risk for heart disease based on a model that uses their diet and workout frequency as attributes. Although most people who eat healthily and exercise regularly have less chance of developing heart disease, they may still be at risk due to other *latent factors*, such as heredity, excessive smoking, and alcohol abuse, that are not captured in the model. Third, a classification model learned over a finite training set may not be able to fully capture the true relationships in the overall data, as discussed in the context of model overfitting in Chapter 3. Finally, uncertainty in predictions may arise due to the inherent *random nature* of real-world systems, such as those encountered in weather forecasting problems.

In the presence of uncertainty, there is a need to not only make predictions of class labels but also provide a measure of confidence associated with every prediction. Probability theory offers a systematic way for quantifying and manipulating uncertainty in data, and thus, is an appealing framework for assessing the confidence of predictions. Classification models that make use of probability theory to represent the relationship between attributes and class labels are known as **probabilistic classification models**. In this section, we present the naïve Bayes classifier, which is one of the simplest and most widely-used probabilistic classification models.

an anomaly. Otherwise, we can use the inverse of the height (frequency) of the bin in which it falls as its anomaly score. This approach is known as the frequency-based or counting-based approach for anomaly detection.

A key step in using frequency-based approaches for anomaly detection is choosing the size of the bin used for constructing the histogram. A small bin size can falsely identify many normal instances as anomalous, since they might fall in empty or sparsely populated bins. On the other hand, if the bin size is too large, many anomalous instances may fall in heavily populated bins and go unnoticed. Thus, choosing the right bin size is challenging, and often requires trying multiple size options or using expert knowledge.

9.3.3 Modeling Normal and Anomalous Classes

The statistical approaches described so far only model the distribution of the normal class but not the anomalous class. They assume that the training set predominantly has normal instances. However, if there are outliers present in the training data, which is common in most practical applications, the learning of the probability distributions corresponding to the normal class may be distorted, resulting in poor identification of anomalies.

Here, we present a statistical approach for anomaly detection that can tolerate a considerable fraction (λ) of outliers in the training set, provided that the outliers are uniformly distributed (and thus not clustered) in the attribute space. This approach makes use of a mixture modeling technique to learn the distribution of normal and anomalous classes. This approach is similar to the Expectation-Maximization (EM) based technique introduced in the context of clustering in Chapter 8.2.2. Note that λ), the fraction of outliers, is like a prior.

The basic idea of this approach is to assume that instances are generated with probability λ from the anomalous class, which has uniform distribution, p_A, and with probability $1 - \lambda$ and from the normal class, which has the distribution, $f_M(\theta)$, where θ represents the parameters of the distribution. The approach for assigning training instances to the normal and anomaly classes can be described as follows. Initially, all the objects are assigned to the normal class and the set of anomalous objects is empty. At every iteration of the EM algorithm, objects are transferred from the normal class to the anomaly class to improve the likelihood of the overall data. Let M_t and A_t be the set of normal and anomalous objects, respectively, at iteration t. The likelihood of the data set D, $\mathcal{L}_t(D)$, and its log-likelihood, $\log\mathcal{L}_t(D)$, are then given by the following equations:

$$\mathcal{L}_t(D) = \prod_{\mathbf{x}_i \in D} P(\mathbf{x}_i) = \left((1-\lambda)^{|M_t|} \prod_{\mathbf{x}_i \in M_t} P_M(\mathbf{x}_i, \theta_t)\right)\left(\lambda^{|A_t|} \prod_{\mathbf{x}_i \in A_t} P_A(\mathbf{x}_i)\right) \quad (9.5)$$

$$\log\mathcal{L}_t(D) = |M_t|\log(1-\lambda) + \sum_{\mathbf{x}_i \in M_t} \log P_M(\mathbf{x}_i, \theta_t) + |A_t|\log\lambda + \sum_{\mathbf{x}_i \in A_t} \log P_A(\mathbf{x}_i) \quad (9.6)$$

where $|M_t|$ and $|A_t|$ are the number of objects in the normal and anomaly classes, respectively, and θ_t represents the parameters of the distribution of the normal class, which can be estimated using M_t. If the transfer of an object $\mathbf{x}$ from M_t to A_t results in a significant increase in the log-likelihood of the data (greater than a threshold c), then $\mathbf{x}$ is assigned to the set of outliers A_t. The set of outliers A_t keeps growing till we achieve the maximum likelihood of the data using M_t and A_t. This approach is summarized in Algorithm 9.1.

Algorithm 9.1 Likelihood-based outlier detection.

1: Initialization: At time $t = 0$, let M_t contain all the objects, while A_t is empty.
2: **for** each object $\mathbf{x}$ that belongs to M_t **do**
3: Move $\mathbf{x}$ from M_t to A_t to produce the new data sets A_{t+1} and M_{t+1}.
4: Compute the new log-likelihood of D, $\log\mathcal{L}_{t+1}(D)$
5: Compute the difference, $\Delta = \log\mathcal{L}_{t+1}(D) - \log\mathcal{L}_t(D)$
6: **if** $\Delta > c$, where c is some threshold **then**
7: Classify $\mathbf{x}$ as an anomaly.
8: Increment t by one and use M_{t+1} and A_{t+1} in the next iteration.
9: **end if**
10: **end for**

Because the number of normal objects is large compared to the number of anomalies, the distribution of the normal objects may not change much when an object is moved to the set of anomalies. In that case, the contribution of each normal object to the overall likelihood of the normal objects will remain relatively constant. Furthermore, each object moved to the set of anomalies contributes a fixed amount to the likelihood of the anomalies. Thus, the overall change in the total likelihood of the data when an object is moved to the set of anomalies is roughly equal to the probability of the object under a uniform distribution (weighted by λ) minus the probability of the object under the distribution of the normal data objects (weighted by $1-\lambda$). Consequently, the set of anomalies will tend to consist of those objects that have significantly higher probability under a uniform distribution than under the distribution of the normal objects.

In the situation just discussed, the approach described by Algorithm 9.1 is roughly equivalent to classifying objects with a low probability under the distribution of normal objects as outliers. For example, when applied to the points in Figure 9.3, this technique would classify points A and B (and other points far from the mean) as outliers. However, if the distribution of the normal objects changes significantly as anomalies are removed or the distribution of the anomalies can be modeled in a more sophisticated manner, then the results produced by this approach will be different than the results of simply classifying low-probability objects as outliers. Also, this approach can work even when the distribution of normal objects is multi-modal, e.g., by using a mixture of Gaussian distributions for $f_M(\theta)$. Also, conceptually, it should be possible to use this approach with distributions other than Gaussian.

9.3.4 Assessing Statistical Significance

Statistical approaches provide a way to assign a measure of confidence for the instances detected as anomalies. For example, since the anomaly scores computed by statistical approaches have a probabilistic meaning, we can apply a threshold to these scores with statistical guarantees. Alternatively, it is possible to define statistical tests (also termed as **discordancy tests**) that can identify the statistical significance of an instance being identified as an anomaly by a statistical approach. Many of these discordancy tests are highly specialized and assume a level of statistical knowledge beyond the scope of this text. Thus, we illustrate the basic ideas with a simple example that uses univariate Gaussian distributions, and refer the reader to the Bibliographic Notes for further pointers.

Consider the Gaussian distribution $N(0, 1)$ shown in Figure 9.1. As discussed previously in Section 9.3.1, most of the probability density is centered around zero and there is little probability that an object (value) belonging to $N(0, 1)$ will occur in the tails of the distribution. For instance, there is only a probability of 0.0027 that an object lies beyond the central area between ± 3 standard deviations. More generally, if c is a constant and x is the attribute value of an object, then the probability that $|x| \geq c$ decreases rapidly as c increases. Let $\alpha = prob(|x| \geq c)$. Table 9.1 shows some sample values for c and the corresponding values for α when the distribution is $N(0, 1)$. Note that a value that is more than 4 standard deviations from the mean is a one-in-ten-thousand occurrence.

This interpretation of the distance of a point from the center can be used as the basis of a test to assess whether an object is an outlier, using the following definition.

Table 9.1. Sample pairs (c, α), $\alpha = prob(|x| \geq c)$ for a Gaussian distribution with mean 0 and standard deviation 1.

c	α for $N(0,1)$
1.00	0.3173
1.50	0.1336
2.00	0.0455
2.50	0.0124
3.00	0.0027
3.50	0.0005
4.00	0.0001

Definition 9.2 (Outlier for a Single *N(0,1)* Gaussian Attribute)**.** An object with attribute value x from a Gaussian distribution with mean of 0 and standard deviation 1 is an outlier if

$$|x| \geq c, \tag{9.7}$$

where c is a constant chosen so that $P(|x| \geq c) = \alpha$, where P represents probability.

To use this definition it is necessary to specify a value for α. From the viewpoint that unusual values (objects) indicate a value from a different distribution, α indicates the probability that we mistakenly classify a value from the given distribution as an outlier. From the viewpoint that an outlier is a rare value of a $N(0, 1)$ distribution, α specifies the degree of rareness.

More generally, for a Gaussian distribution with mean μ and standard deviation σ, we can first compute the z score of x and then apply the above test on x. In practice, this works well when μ and σ are estimated from a large population. A more sophisticated statistical procedure (Grubbs' test), which takes into account the distortion of parameter estimates caused by outliers, is explored in Exercise 7 on page 770.

The approach to outlier detection presented here is equivalent to testing data objects for statistical significance and classifying the statistically significant objects as anomalies. This is discussed in more detail in Chapter 10.

9.3.5 Strengths and Weaknesses

Statistical approaches to outlier detection have a firm theoretical foundation and build on standard statistical techniques. When there is sufficient knowledge of the data and the type of test that should be applied, these approaches are statistically justifiable and can be very effective. They can also provide